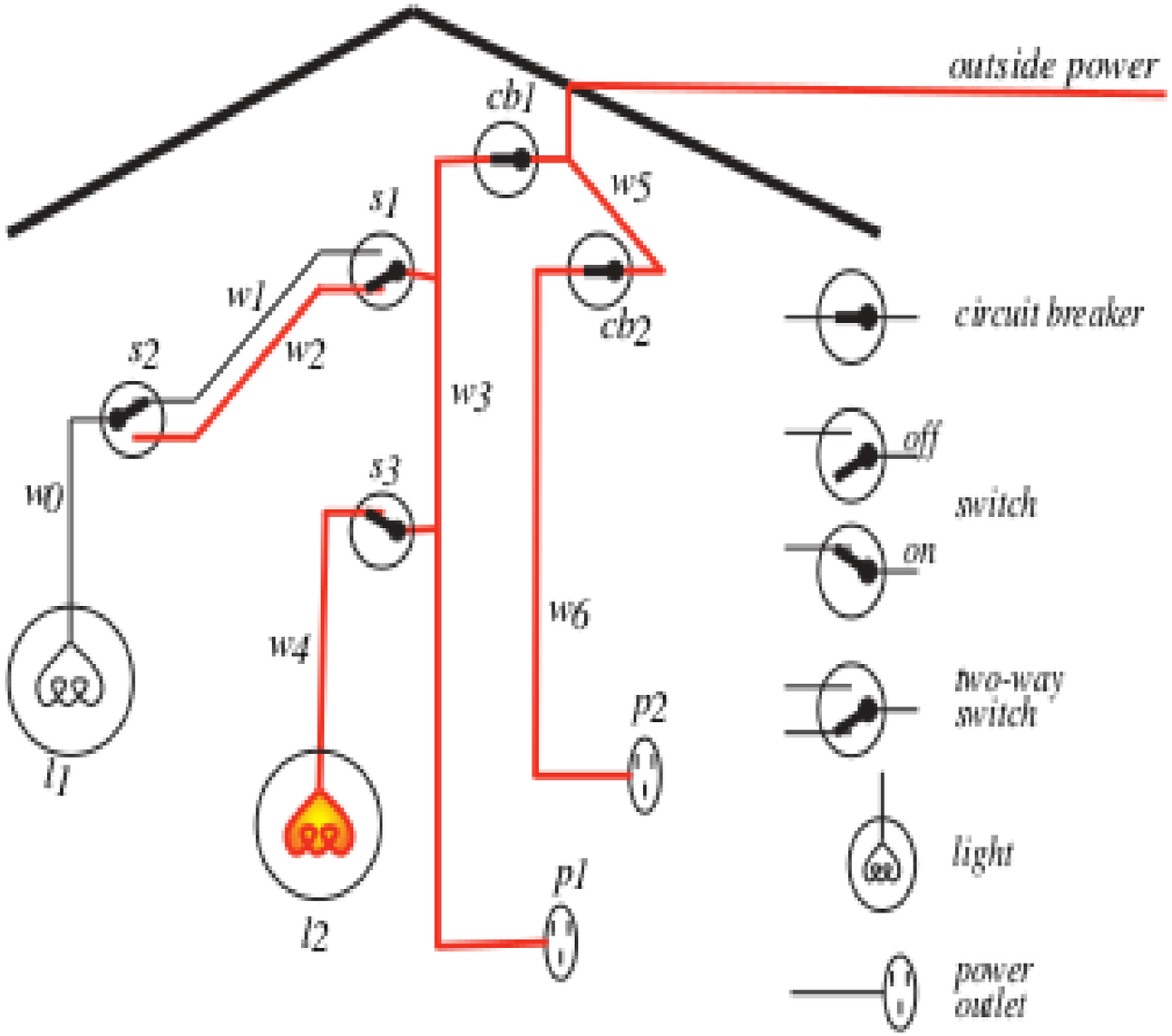
UCS3461 Foundations of Artificial Intelligence

LOGICAL AGENTS

Semester 4, B.E. Computer Science and Engineering
23 January 2026

Sakaya Milton R
SSN College of Engineering

Electrical Environment KB Facts



$light_l_1.$

$light_l_2.$

$down_s_1.$

$up_s_2.$

$up_s_3.$

$ok_l_1.$

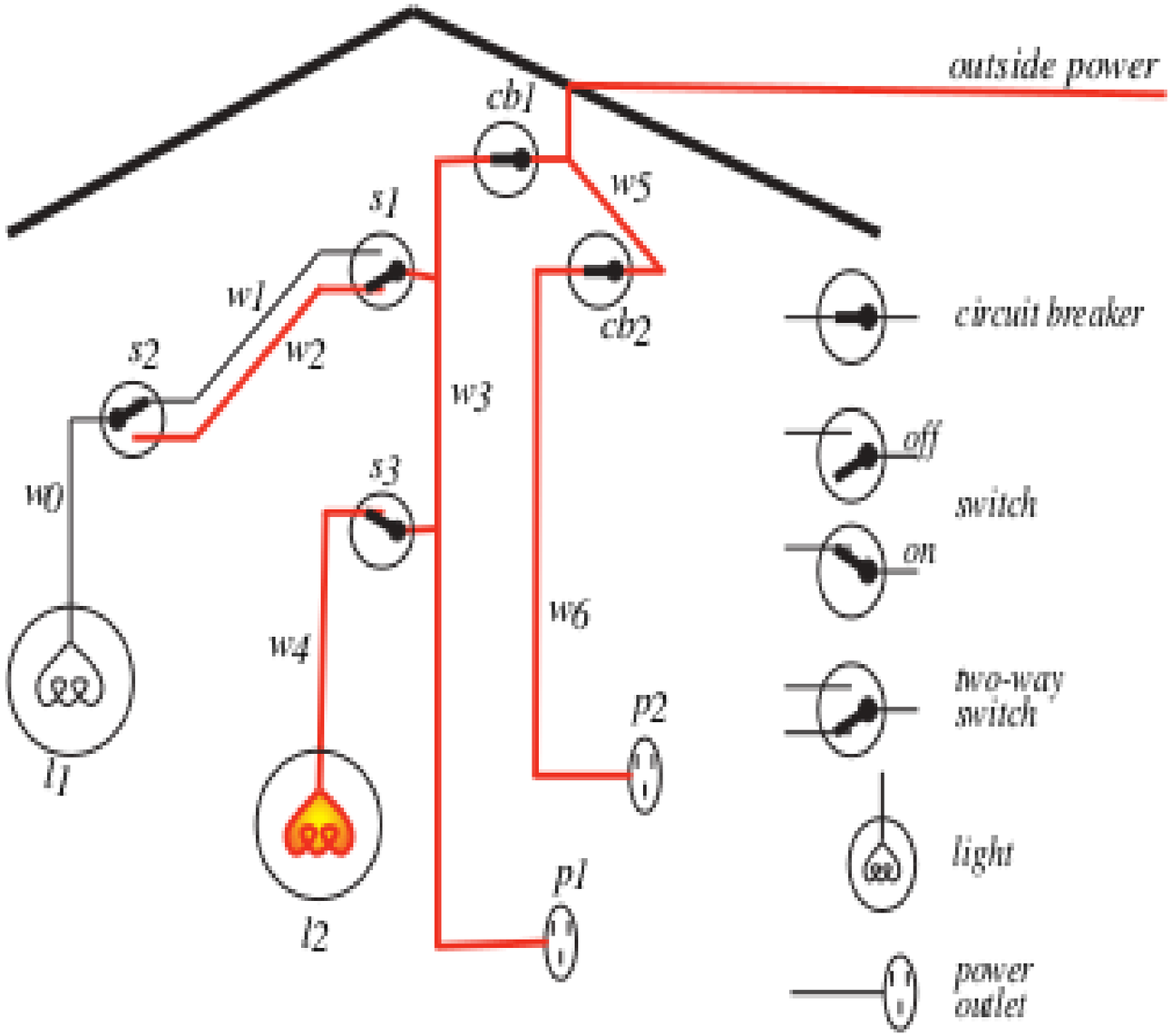
$ok_l_2.$

$ok_cb_1.$

$ok_cb_2.$

$live_outside.$

Electrical Environment KB Rules



$lit_l_1 \leftarrow live_w0 \wedge ok_l_1$

$live_w_0 \leftarrow live_w_1 \wedge up_s_2.$

$live_w_0 \leftarrow live_w_2 \wedge down_s_2.$

$live_w_1 \leftarrow live_w_3 \wedge up_s_1.$

$live_w_2 \leftarrow live_w_3 \wedge down_s_1.$

$lit_l_2 \leftarrow live_w_4 \wedge ok_l_2.$

$live_w_4 \leftarrow live_w_3 \wedge up_s_3.$

$live_p_1 \leftarrow live_w_3.$

$live_w_3 \leftarrow live_w_5 \wedge ok_cb_1.$

$live_p_2 \leftarrow live_w_6.$

$live_w_6 \leftarrow live_w_5 \wedge ok_cb_2.$

$live_w_5 \leftarrow live_outside.$

Queries and Answers

ask $light_l_1$.

answer: yes.

ask $light_l_6$.

answer: no.

ask lit_l_2

answer: yes.

Propositional Logic

Propositonal Logic

- A proposition is a sentence that has a truth value – it is true or false – in a world.
- An atomic proposition or atom: symbol
- A compound proposition is built from atomic propositions (atoms) and logical connectives.
- Proposition in a knowledge representation language is
  - an atomic proposition, or
  - a compound proposition

Syntax

- Propositions are sentences in knowledge representation language
- Atomic propositions are symbols
- Sentences have a structure (grammar)
- Logical connectives: Let A and B be propositions. Then $(\neg A)$, $(A \wedge B)$, $(A \vee B)$, $(A \rightarrow B)$, $(A \leftrightarrow B)$ are propositions.

Connective	Proposition	Read as
negation	$\neg A$	not A
conjunction	$A \wedge B$	A and B
disjunction	$A \vee B$	A or B
conditional	$A \rightarrow B$	if A then B
biconditional	$A \leftrightarrow B$	A if and only if B

- Propositions or propositional formulas or formulas

Semantics

- True and False are truth values, denoted by T and F, respectively.
- Truth values of formulas.
- Truth tables for logical connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

Interpretation

- An interpretation is an assignment of truth values to each atom in a formula.
- Interpretation, assignment, state
- An interpretation is defined by function I that maps atoms to $\{$True, False$\}$.
  - $I(a) =$ True: atom a is True in the interpretation.
  - $I(a) =$ False: atom a is False in the interpretation.
- Truth of a compound proposition in an interpretation can be evaluated recursively from the truth of its components.
- Propositions can have different truth values in different interpretations.

Interpretation: Example

- $\neg a \rightarrow b \wedge c \equiv (\neg a) \rightarrow (b \wedge c)$ (precedence of connectives)
- Truth tables of logical connectives

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \rightarrow Q$	$P \leftrightarrow Q$
F	F	T	F	F	T	T
F	T	T	F	T	T	F
T	F	F	F	T	F	F
T	T	F	T	T	T	T

- Interpretation: $I(a) = T, I(b) = F, I(c) = T$

$$\begin{aligned}
&\neg a \rightarrow b \wedge c \\
=&(\neg a) \rightarrow (b \wedge c) \\
=&(\neg T) \rightarrow (F \wedge T) \\
=&F \rightarrow (F \wedge T) \\
=&F \rightarrow F \\
=&T
\end{aligned}$$

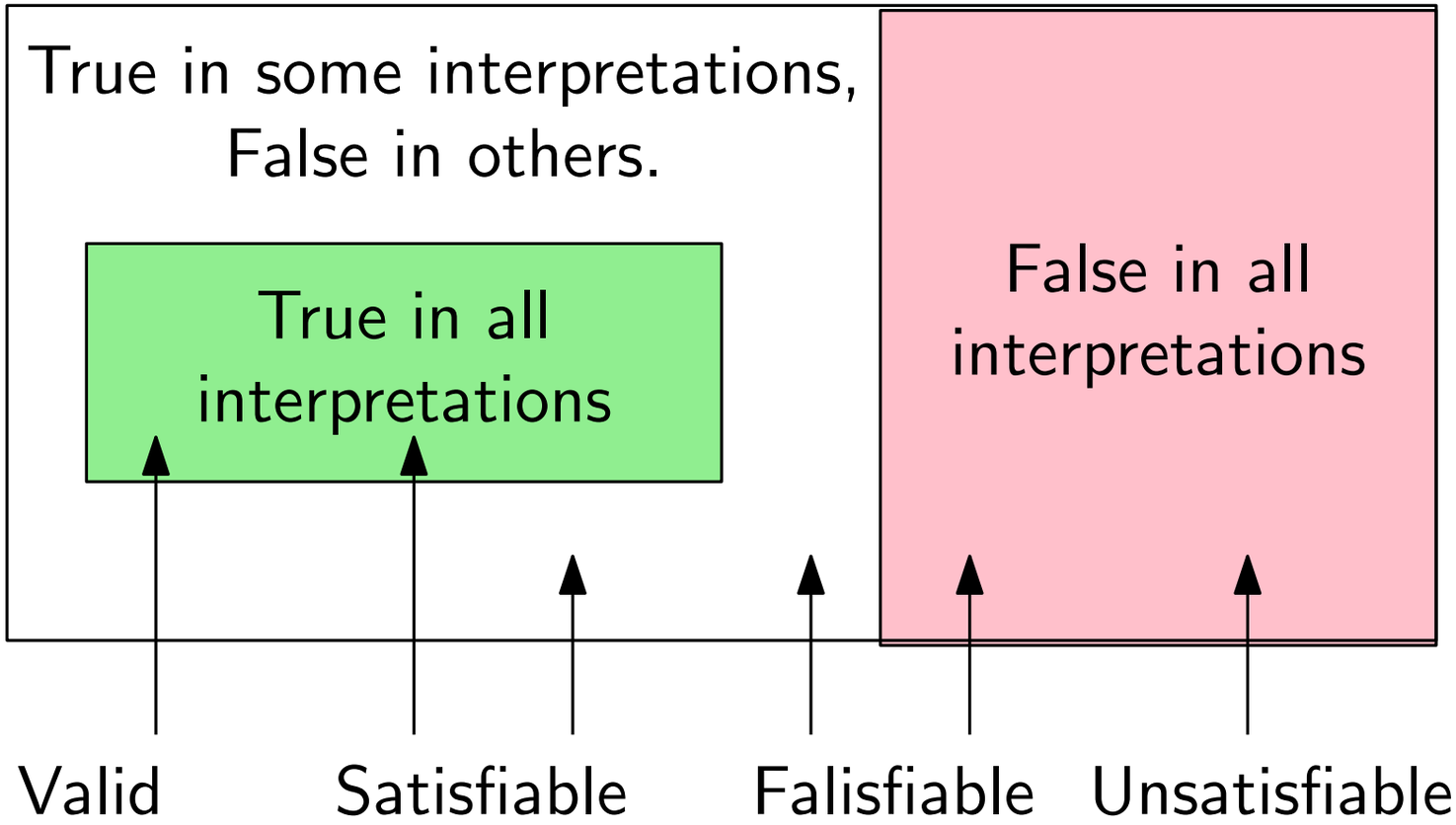
Validity and Unsatisfiability

- A formula A is satisfiable iff there is at least one interpretation I in which A evaluates to T: $I(A) = T$
- A formula A is valid iff A evaluates to T in every interpretation (always true, tautology)
- A formula A is unsatisfiable iff A evaluates to F in every interpretation (always false, contradiction)

Model

- A satisfying interpretation I is a model of a formula A $I(A) = T$.
- A satisfiable formula has at least one model.
- Every interpretation is a model for a A valid formula
- An unsatisfiable formula has no model.

Validity and Unsatisfiability



Validity and Unsatisfiability: Example

$a \wedge b \leftrightarrow \neg a \vee \neg b$ is unsatisfiable

a	b	$a \wedge b$	$\neg a$	$\neg b$	$\neg a \vee \neg b$	$a \wedge b \leftrightarrow \neg a \vee \neg b$
F	F	F	T	T	T	F
F	T	F	T	F	T	F
T	F	F	F	T	T	F
T	T	T	F	F	F	F

$\neg(a \wedge b) \leftrightarrow \neg a \vee \neg b$ is valid

Entailment

- An interpretation assigns a truth value to every atom in a formula
  - it is a function from atoms to truth values,
  - extended to be a function from formulas to truth values.
- Let A and B be formulas. If, in every interpretation in which A is true, B is also true, then A entails B.
  - B logically follows A
  - B is a logical consequence of A
- $A \models B$
- In every interpretation I in which $I(A) = T$, $I(B) = T$.
- $I(A \rightarrow B) = T$ in every interpretation I.
- $A \models B$ iff $A \rightarrow B$ is valid.
- Every model of A is a model of B.

Entailment: Example

Show that $a \wedge a \rightarrow b \models b$

a	b	$a \rightarrow b$	$a \wedge a \rightarrow b$	$(a \wedge a \rightarrow b) \rightarrow b$
F	F	T	F	T
F	T	T	F	T
T	F	F	F	T
T	$\boxed{T}$	T	$\boxed{T}$	T

Show that

$a \rightarrow b \wedge b \rightarrow c \models a \rightarrow c$

Equivalence

- Two formulas A and B are equivalent to each other iff, in every interpretation, A and B evaluate to identical truth values.
- A is equivalent to B
- $A \equiv B$
- $I(A) = I(B)$ in every interpretation I suitable for A and B
- $I(A \leftrightarrow B) = T$ in every interpretation I.
- $A \equiv B$ iff $A \leftrightarrow B$ is valid.
- A and B have the same set of models.

Equivalence: Example

Show that $a \rightarrow b \equiv \neg a \vee b$

a	b	$a \rightarrow b$	$\neg a$	$\neg a \vee b$	$(a \rightarrow b) \leftrightarrow (\neg a \vee b)$
F	F	T	T	T	T
F	T	T	T	T	T
T	F	F	F	F	T
T	$\boxed{T}$	T	F	T	T

Show that

$a \leftrightarrow b \equiv (a \rightarrow b) \wedge (b \rightarrow a)$

Validity, Unsatisfiability, Entailment, Equivalence

- An interpretation I assigns a truth value to every atom in a formula A.
- A formula A is satisfiable iff there is an interpretation I in which $I(A) = T$.
- A formula A is valid iff $I(A) = T$ in every interpretation I
- A formula A is unsatisfiable iff $I(A) = F$ in every interpretation I
- Formula A entails a formula B iff, in every interpretation I in which $I(A) = T$, $I(B) = T$.

$$A \models B$$

- Two formulas A and B are equivalent to each other iff $I(A) = I(B)$ in every interpretation suitable for A and B, written as

$$A \equiv B$$

Formulas in Normal Form

Conjunctive Normal Form (CNF)

- Literal: an atom or the negation of an atom
- A conjunction of disjunction of literals

$$(\neg p \vee q \vee r) \wedge (\neg q \vee r) \wedge (\neg r)$$

Conversion of a Formula to Equivalent Formula in CNF

- Every PL formula can be transformed into an equivalent PL formula in CNF.
- Steps
  1. Eliminate $\leftrightarrow$
     $a \leftrightarrow b \equiv a \rightarrow b \wedge b \rightarrow a$
  2. Eliminate $\rightarrow$
     $a \rightarrow b \equiv \neg a \vee b$
  3. "Move $\neg$ inwards" by applying De Morgan's laws
     $\neg(a \vee b) \equiv \neg a \wedge \neg b, \neg(a \wedge b) \equiv \neg a \vee \neg b$
  4. Eliminate double negation
     $\neg\neg a \equiv a$
  5. Apply distribuitive laws
     $a \wedge (b \vee c) \equiv (a \wedge b) \vee (a \wedge c), a \vee (b \wedge c) \equiv (a \vee b) \wedge (a \vee c)$

Conversion to CNF: Example

$$(\neg p \rightarrow \neg q) \rightarrow (p \rightarrow q)$$

Clausal Form

- A literal is an atom (positive literal) or the negation of an atom (negative literal).
- A clause is a set of literals (implicit disjunction of its literals).
- A formula in clausal form is a set of clauses (implicit conjunction of its clauses).
- An empty set of literals is the empty clause, denoted by $\square$. An empty clause is false.
- The formula that is the empty set of clauses is denoted by $\emptyset$. An empty clause set is true.

Conversion to Clausal Form: Example

$$(p \vee r) \wedge (\neg q \vee \neg p \vee q) \wedge (p \vee \neg p \vee q \vee p \vee \neg p) \wedge (r \vee p)$$

$$\{\{p, r\}, \{\neg q, \neg p, q\}, \{p, \neg p, q\}, \{r, p\}\}$$

Empty Clause is False

$$(\neg p \rightarrow \neg q) \rightarrow (p \rightarrow q)$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$

$$\equiv (\neg\neg p \vee \neg q) \vee (\neg p \vee q)$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$

$$\equiv (\neg\neg p \vee \neg q) \vee (\neg p \vee q)$$

$$\equiv p \vee \neg q \vee \neg p \vee q$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$

$$\equiv (\neg\neg p \vee \neg q) \vee (\neg p \vee q)$$

$$\equiv p \vee \neg q \vee \neg p \vee q$$

$$\equiv p \vee \neg p \vee q \vee \neg q$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$

$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$

$$\equiv (\neg\neg p \vee \neg q) \vee (\neg p \vee q)$$

$$\equiv p \vee \neg q \vee \neg p \vee q$$

$$\equiv p \vee \neg p \vee q \vee \neg q$$

$$\equiv F \vee F$$

Empty Clause is False

$$(\neg p \to \neg q) \to (p \to q)$$

$$(\neg p \to \neg q) \to (p \to q)$$
$$\equiv \neg(\neg p \to \neg q) \vee (p \to q)$$
$$\equiv (\neg\neg p \vee \neg q) \vee (\neg p \vee q)$$
$$\equiv p \vee \neg q \vee \neg p \vee q$$
$$\equiv p \vee \neg p \vee q \vee \neg q$$
$$\equiv F \vee F$$
$$\equiv F$$

$$\{p, \neg p, q, \neg q\}$$
$$\equiv \{\}$$
$$\equiv \square$$

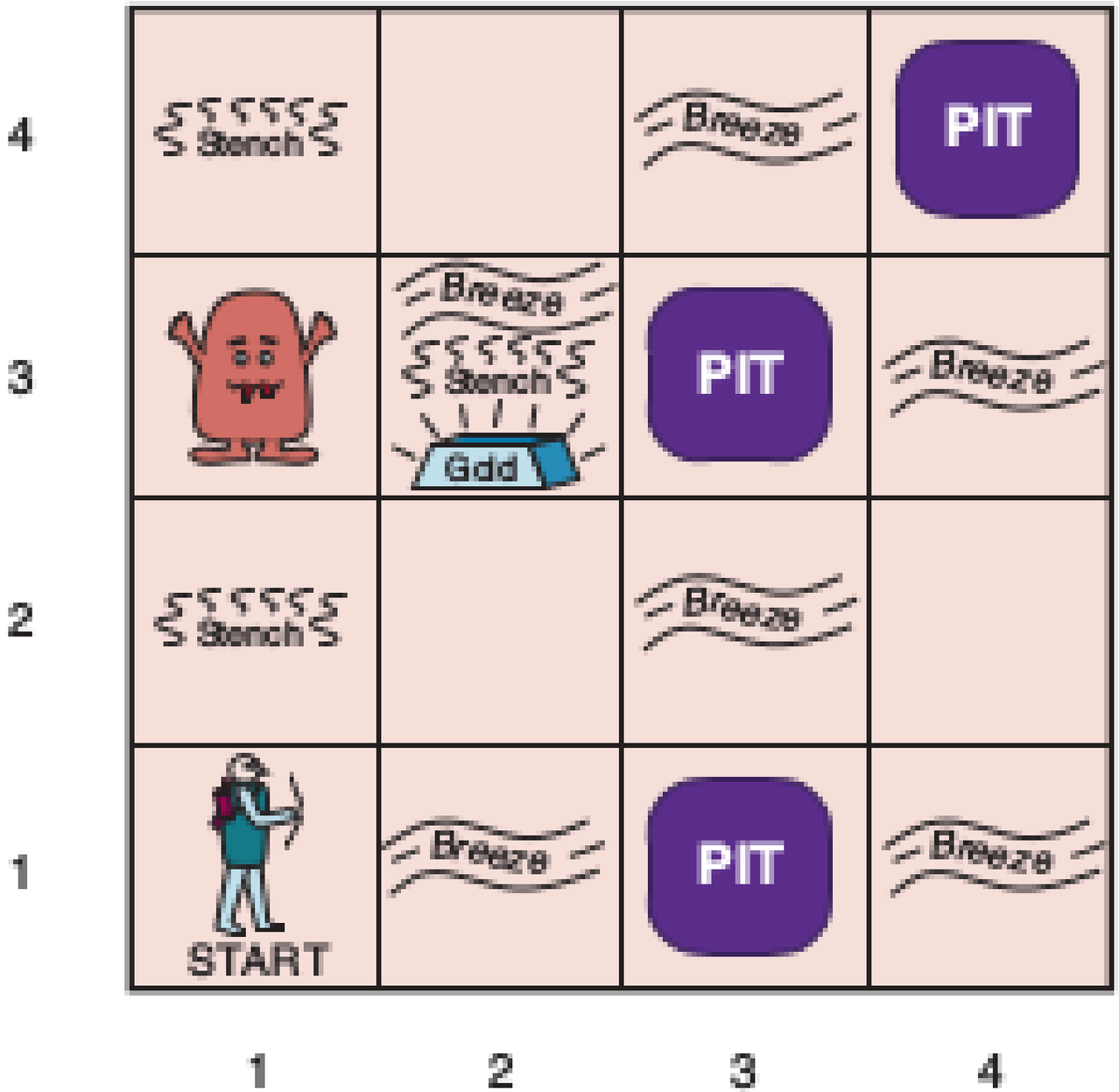
Knowledge Base

Knowledge Base

- A Knowledge Base is a set of formulas.
- Each formula in the knowledge base is called an axiom
- Axioms of a knowledge base are conjoined – a knowledge base is true iff all its axioms are true.
- If every axiom is a clause, then the knowledge base is a clause set (a formula in CNF).
- M is a model of a KB iff M is a model for every axiom of the KB.
- Let K be a knowledge base (a set of formulas) and G be a formula. If every model of K is a model of G, $K \models G$.

$$\neg P_{11}$$
$$B_{11} \leftrightarrow P_{12} \vee P_{21}$$
$$B_{21} \leftrightarrow P_{11} \vee P_{22} \vee P_{31}$$
$$\neg B_{11}$$
$$B_{21}$$

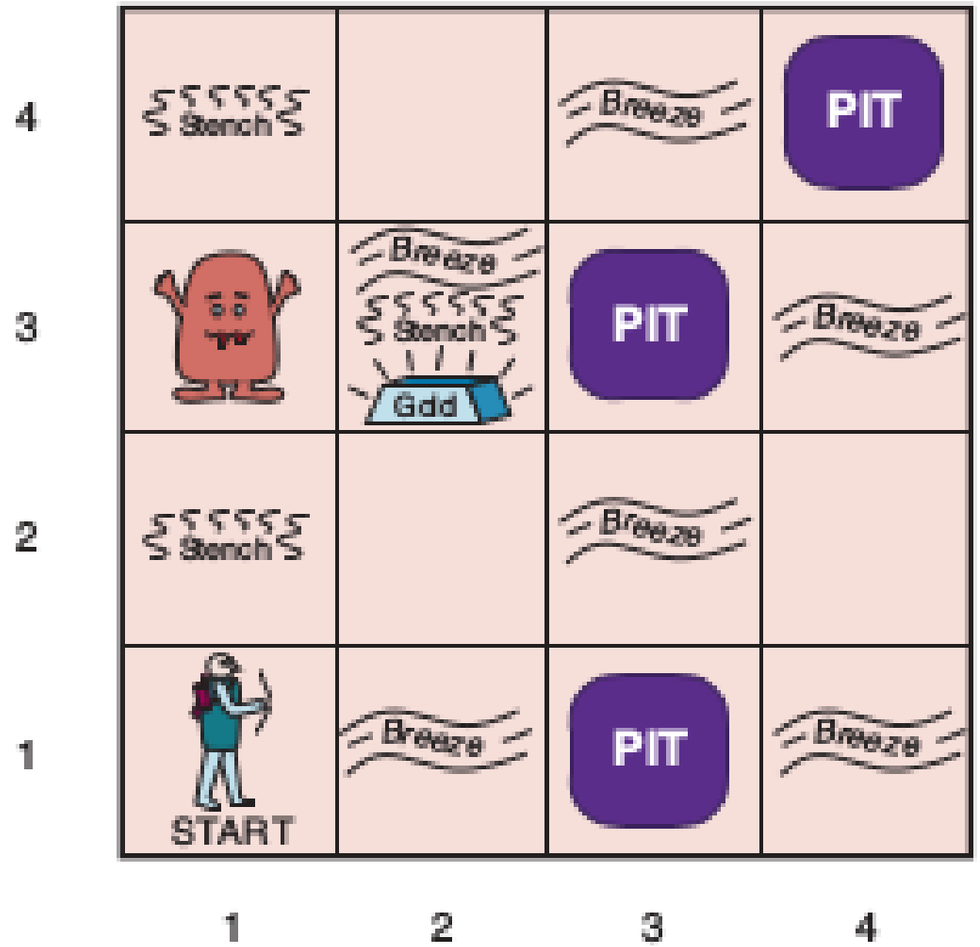
Knowledge Base: Wumpus World Example



- Stench in a square if (vertically or horizontally) adjacent to the Wumpus cell.
- Breeze in a square if (vertically or horizontally) adjacent to the PIT cell.

Knowledge Base: Wumpus World Example

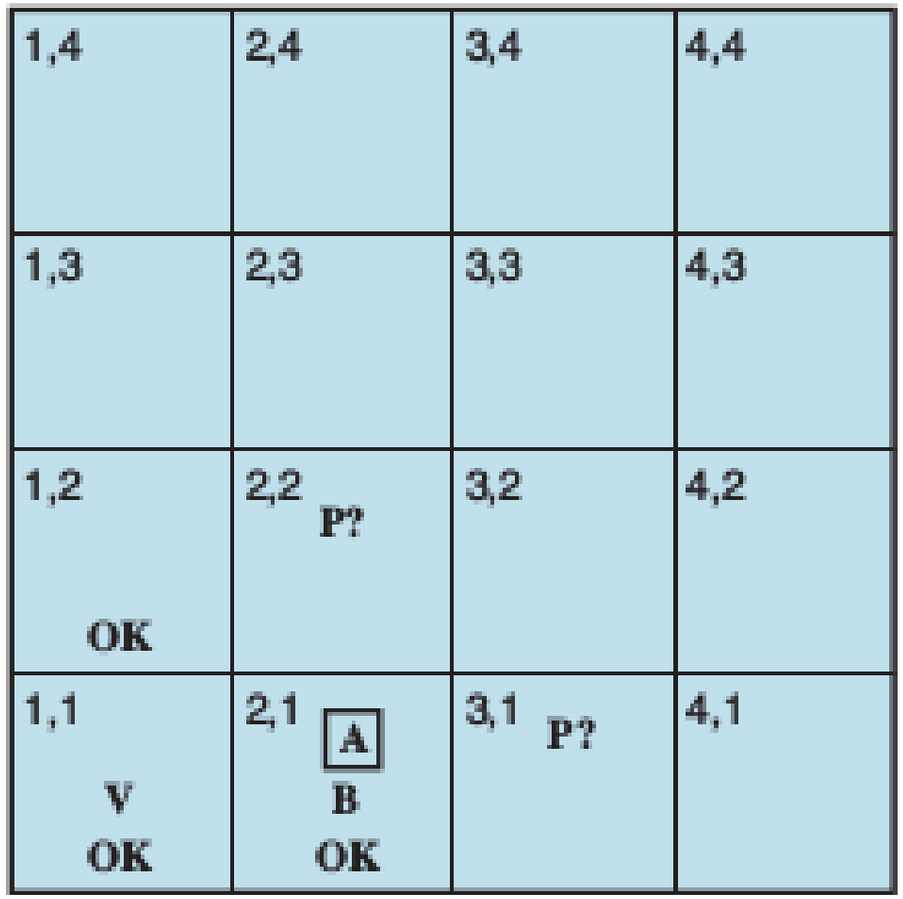
- Performance measure: $+1000$ gold, -1000 dead, -1 each action, -10 use an arrow
- Environment: 4×4 grid, start at $(1, 1)$, gold and wumpus location random, pit probability $= 0.2$
- Actuators: forward, turn left by 90, turn right by 90, grab, shoot, climb
- Sensors: stench (from adjacent squares of the wumpus), breeze, glitter, bump (into a wall), scream (from wumpus when dead)



- Sensor $=$ (stench, breeze, glitter, bump, scream)

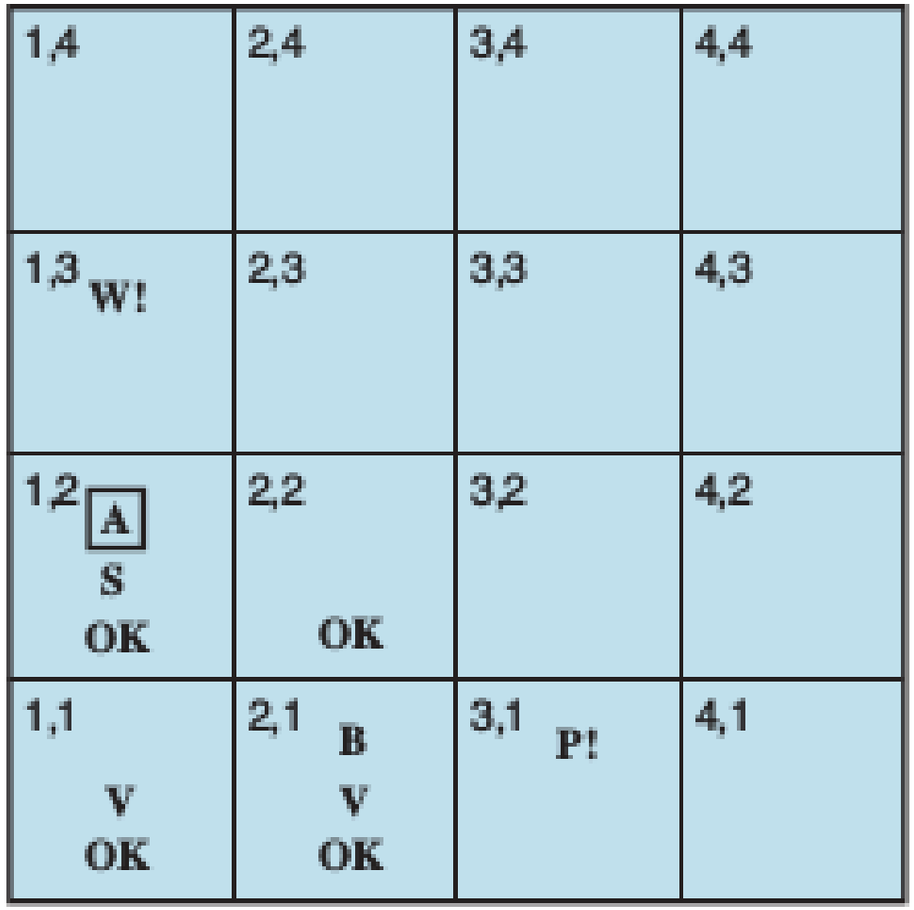
Knowledge Base: Wumpus World Example

A = *Agent*
B = *Breeze*
G = *Glitter, Gold*
OK = *Safe square*
P = *Pit*
S = *Stench*
V = *Visited*
W = *Wumpus*

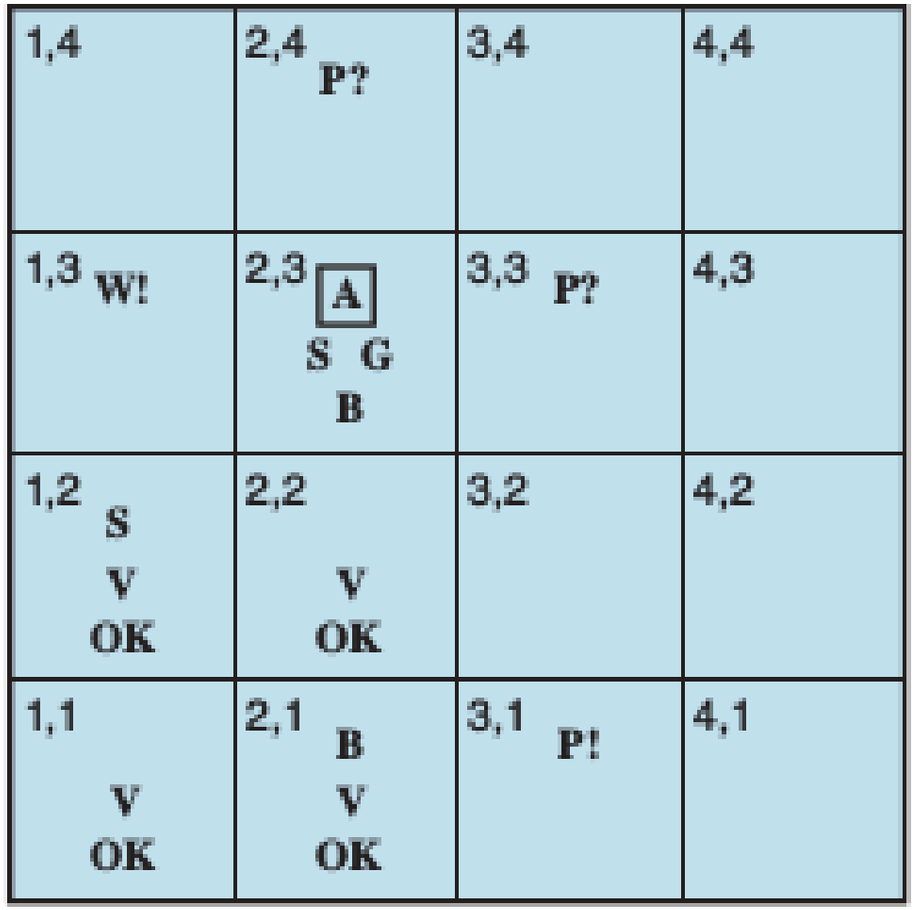


- Initial situation, after percept (None, None, None, None, None)
- After one move, with percept (None, Breeze, None, None, None)

Knowledge Base: Wumpus World Example



A = *Agent*
B = *Breeze*
G = *Glitter, Gold*
OK = *Safe square*
P = *Pit*
S = *Stench*
V = *Visited*
W = *Wumpus*



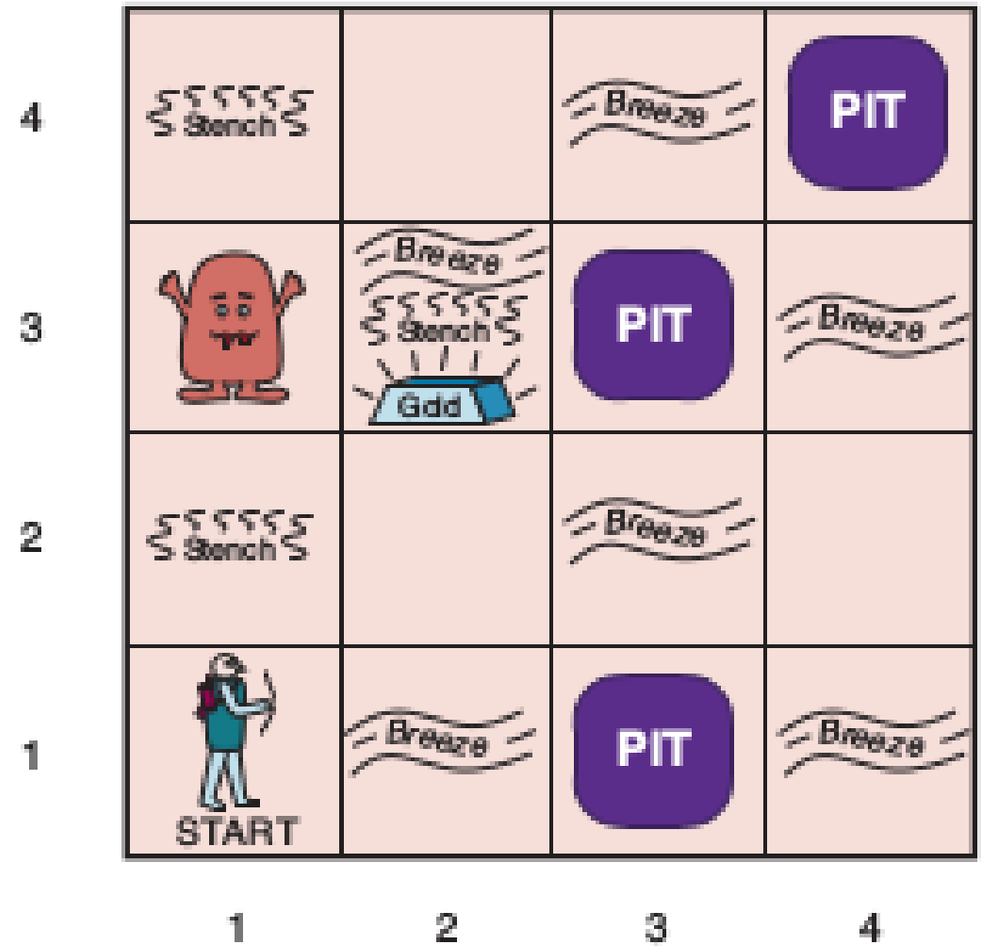
- After the third move, with percept (Stench, None, None, None, None)
- After the fifth move, with percept (Stench, Breeze, Glitter, None, None)

Knowledge Base and Query: Example

P_{xy}: There is a pit in (x, y).
W_{xy}: There is a wumpus in (x, y), dead or alive.
B_{xy}: Breeze perceived in (x, y).
S_{xy}: Stench perceived in (x, y).

KB

$$R_1 : \neg P_{11}$$
$$R_2 : B_{11} \leftrightarrow P_{12} \vee P_{21}$$
$$R_3 : B_{21} \leftrightarrow P_{11} \vee P_{22} \vee P_{31}$$
$$R_4 : \neg B_{11}$$
$$R_5 : B_{21}$$



Decide whether $KB \models \neg P_{12}$

Model Checking

Inference by Model Checking

- Atoms

  P_{xy}: There is a pit in (x, y).
  W_{xy}: There is a wumpus in (x, y), dead or alive.
  B_{xy}: Breeze perceived in (x, y).
  S_{xy}: Stench perceived in (x, y).

- K

$$R_1 : \neg P_{11}$$
$$R_2 : B_{11} \leftrightarrow P_{12} \vee P_{21}$$
$$R_3 : B_{21} \leftrightarrow P_{11} \vee P_{22} \vee P_{31}$$
$$R_4 : \neg B_{11}$$
$$R_5 : B_{21}$$

- Decide whether $K \models G$
- Decide whether $K \models \neg P_{12}$

Model Checking Procedure

	B_{11}	B_{21}	P_{11}	P_{12}	P_{21}	P_{22}	P_{31}	R_1	R_2	R_3	R_4	R_5	KB
1	F	F	F	F	F	F	F	F	F	F	F	F	F
2	F	F	F	F	F	F	T	T	T	F	T	F	F
	⋮												
	F	T	F	F	F	F	F	T	T	F	T	T	F
	F	T	F	F	F	F	T	T	T	T	T	T	T
	F	T	F	F	F	T	F	T	T	T	T	T	T
	F	T	F	F	F	T	T	T	T	T	T	T	T
	F	T	F	F	T	F	F	T	F	F	T	T	F
	⋮												
128	T	T	T	T	T	T	T	F	T	T	F	T	F

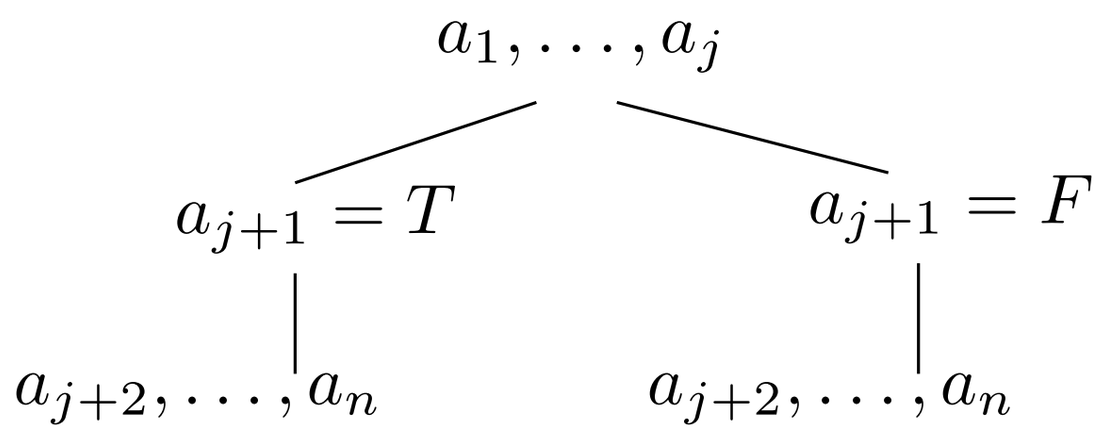
- $2^7 = 128$ rows (assignments)
- 3 rows, R_1 through R_5 are true $\equiv$ KB is true
- In those 3 rows, $\neg P_{12}$ is true
- $KB \models \neg P_{12}$
- $KB \models P_{22}$?

Exhaustive Enumeration

Algorithm: Exhaustive_Enumerate(assign, j)

Input: assign[1 . . . j]
Output: assign[1 . . . n] $\supseteq$ assign[1 . . . j] printed

if j $=$ n **then**
    print assign
    return
assign[j+1] := T
Exhaustive_Enumerate(assign)
assign[j+1] := F
Exhaustive_Enumerate(assign)



Model Checking Algorithm

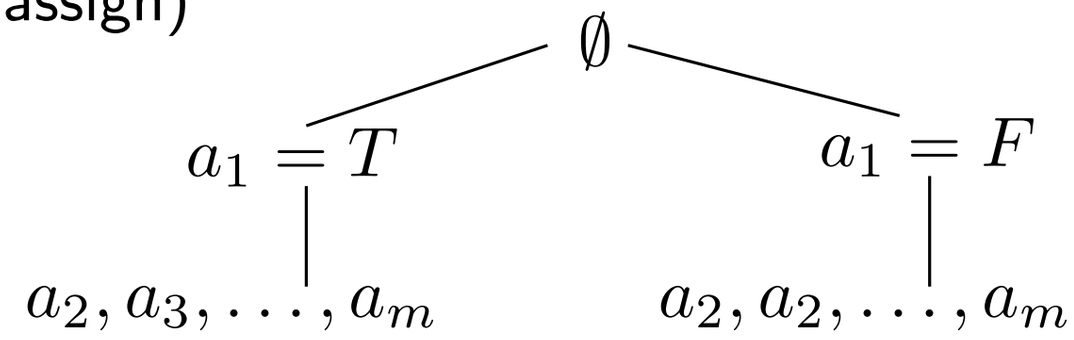
Algorithm: Entails_TT (KB, G, atoms, assign)

Input:

KB: a set of clauses

G: goal clause

Output: $KB \rightarrow G$



if atoms $=$ [ ] **then**

  return Evaluate($KB \rightarrow G$, assign)

a := First(atoms)

rest := Rest(atoms)

val_pos := Entails_TT(KB, G, rest , assign $\cup \{a = T\}$)

val_neg := Entails_TT(KB, G, rest, assign $\cup \{a = F\}$)

return val_pos $\wedge$ val_neg

Analysis of Model Checking Algorithm

Algorithm: Entails_TT_Wrapper(KB, G)

Input:

KB: knowledge base in PL
G: query in PL

Output: $KB \rightarrow G$

atoms := list of atoms in KB and G
return Entails_TT(KB, G, atoms, $\{\}$)

- Sound, complete (finitely many assignments)
- Recursive backtracking search: A has n atoms – 2^n assignments – $O(2^n)$ time-complexity
- $O(n)$ space-complexity
- PL entailment is Co-NP-complete (i.e., probably no easier than NP-complete)
- Every known inference algorithm for PL has a worst-case complexity, exponential in the size of the input.

End